IS5113-Software Quality Assurance

L-02 :Software Quality Factors, McCall Quality Model

W.T.S.Somaweera

Faculty of Computing
Sabaragamuwa University of Sri Lanka

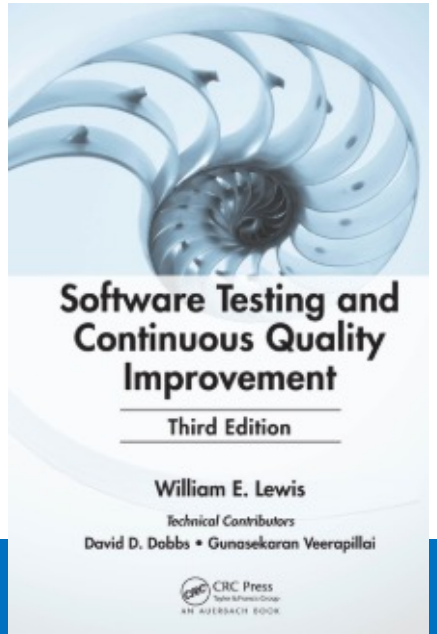
Lecture 02 :Intended Learning Outcomes

- ILO1 : Understand the structure and components of McCall's Quality Model
- ILO2 : Identify and evaluate the 11 quality factors in McCall's Model.
- ILO3 : Apply McCall's Model to assess and improve software quality in real-world projects.

Recommended Reading

- N. Kshirasagar and P. Tripathy, “Software testing & quality assurance: theory & practice”, Sep 2011.
- W. E. Lewis, “Software testing & continuous quality improvement. CRC press”, 2017..
- S. Goericke. “The future of software quality assurance”, Springer Nature, 2020.





Ensuring quality software…

There are multiple aspects to building quality software:

- **Organizational processes** - How does the software team operate?
- **Process and software standards** - Are particular standards used? Process improvement - How is success in building quality software measured and improved?
- **Requirements specification** - How do we work out what software we should be building? And how do we work out whether we built the right software?
- **Formal methods** - Ways of proving that software is correct
- **Testing** - identifying and correcting defects

The software "illities"

Technical Quality Parameters

- **Correctness** - lack of bugs and defects measured in terms of defect rate (# bugs per line of code)
- **Reliability** - does not fail or crash often measured in terms of failure rate (#failures per hour)
- **Capability** - does all that is required measured in terms of requirements coverage (% of required operations implemented)

The software "illities"

Technical Quality Parameters

- **Maintainability** - is easy to change and adapt to new requirements measured in terms of change logs (time and effort required to add a new feature) and impact analysis (#lines affected by a new feature)

- **Performance** - is fast and small enough measured in terms of speed and space usage (seconds of CPU time, Mb of memory, etc.)

User Quality Parameters

- **Usability -** is sufficiently convenient for the intended users measured in terms of user satisfaction (% of users happy with interface and ease of use)
- **Installability -** is convenient and fast to install measured in terms of user satisfaction (#install problems reported per installation)
- **Documentation -** is well documented measured in terms of user satisfaction (% of users happy with documentation)
- **Availability -** is easy to access and available when needed measured in terms of user satisfaction (% of users reporting access problems)

Software Quality Models

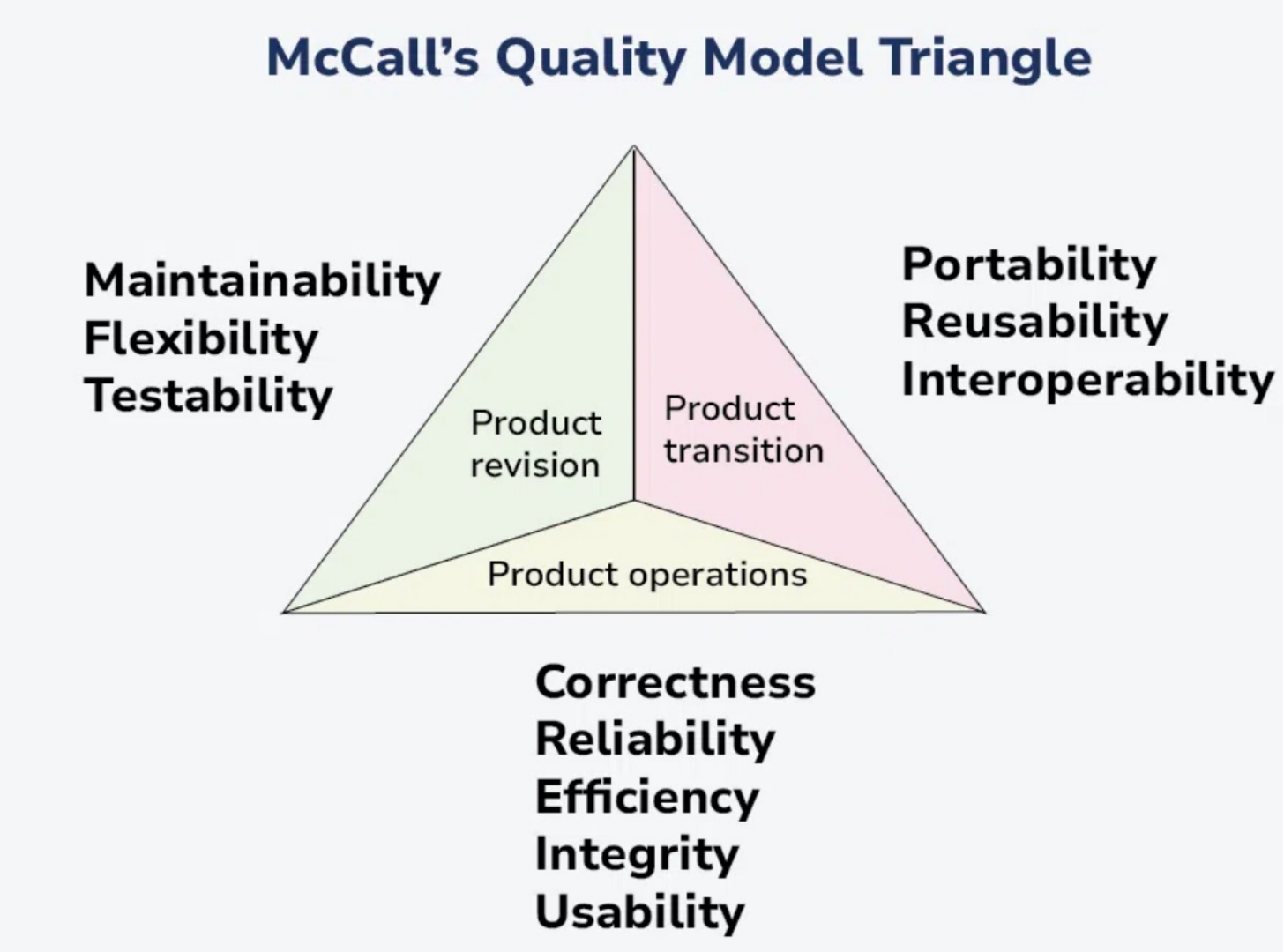
Software Quality Models are **structured frameworks** that define and evaluate the quality attributes of software systems.

These models help guide developers, testers, and stakeholders in measuring, improving, and ensuring software quality throughout the development lifecycle.

Software Quality Models

- **ISO 9126 / 25010**
- **Boehm's Model**
- **McCall's Model** (focus of today's lecture)

McCall's Quality Model



Structure of McCall's Model

Three main aspects:

- **Quality Factors** (external): higher-level quality attributes that can be accessed directly.

- **Quality Criteria** (internal): lower or second-level quality attributes that can be accessed either subjectively or objectively.

- **Metrics** (measurable indicators)

Main Quality Categories / Factors of Product Quality

- **Product Operation :** Focuses on how well the software performs its intended functions during daily use.

- **Product Revision :** Concerned with the ease of making changes to the software after its initial release.

- **Product Transition:** Assesses how well the software can adapt to new environments, systems, or purposes.

Product Operation Factors

Correctness

Measures how well software meets user requirements

Example: *A banking app correctly calculates interest*

Metrics: *Functional completeness, accuracy*

Reliability

Ability to maintain performance under conditions

Examples: *Crash recovery, consistent performance*

Metrics*: Mean time between failures (MTBF)*

Product Operation Factors

Efficiency

Optimal use of resources (CPU, memory, bandwidth)

Example: *Fast response time in high-load scenarios*

Metrics: *Response time, throughput, CPU usage*

Integrity

Protection against unauthorized access or corruption

Examples: Authentication, data encryption

Metrics: Number of security breaches, access violations

Usability

How easy the system is to learn and use

Example: Simple UI for an e-commerce checkout

Metrics: User satisfaction score, training time

Product Revision Factors

Maintainability

Ease with which bugs can be fixed or updates made

Example: Modular code that simplifies updates

Metrics: Mean time to repair (MTTR), defect density

Flexibility

Ease of modifying the software to accommodate changes

Example: Adding new payment gateways to an app

Metrics: Cost of modification, effort required

Testability

Ease of verifying software functions

Example: Availability of test hooks or logs

Metrics: Test coverage, test case execution time

Product Transition Factors

Portability

Ability to run on different environments or platforms

Example: Web app works on Windows, Linux, Mac

Metrics: Number of platforms supported, installation effort

Reusability

Components can be used in different systems

Example: A shared authentication module

Metrics: Reuse ratio, number of reused components

Interoperability

Software's ability to interact with other systems

Example: Integration with third-party APIs

Metrics: Number of interfaces supported, integration success rate

Sample Metrics for Each Quality Factor

Quality Factor	Description	Sample Metric
Correctness	Software performs required functions	✅ **Defect rate** (bugs per KLOC)
Reliability	Consistent performance under conditions	✅ **System uptime %**
Efficiency	Optimal use of system resources	✅ **CPU usage / Response time**
Integrity	Protection from unauthorized access	✅ **Number of security breaches**
Usability	Ease of use and learning	✅ **User satisfaction score**
Maintainability	Ease of identifying and fixing issues	✅ **Mean Time To Repair (MTTR)**
Flexibility	Ease of modifying software	✅ **Effort to implement changes**
Testability	Ease of testing and validation	✅ **Test coverage (%)**
Portability	Ability to run on different platforms	✅ **Number of supported platforms**
Reusability	Reuse of components in other projects	✅ **Component reuse ratio**
Interoperability	Ability to integrate with other systems	✅ **Integration success rate**

Limitations of McCall's Model

- Outdated in terminology (e.g., limited mention of UX, cloud, AI)
- Lack of emphasis on modern agility practices
- Still useful for foundational understanding

Applying McCall's Model in Practice

The city of Greenwood has recently implemented a Hospital Management System (HMS) in its government hospitals to improve patient care and streamline operations. This system handles patient registrations, appointment scheduling, medical records, pharmacy orders, billing, lab results, and doctor-patient interactions.

After the initial deployment, several issues emerged:

- Doctors reported that the system is slow during peak hours.
- Some patients were charged incorrectly due to billing mismatches.
- Nurses found it difficult to navigate the interface, especially during emergencies.
- System crashed once during a surgery schedule update.
- There were security concerns when some staff accessed sensitive records without proper authorization.
- The health ministry plans to extend the same system to rural clinics and integrate it with insurance APIs.

Analyze how each of McCall's 11 software quality factors can be applied to evaluate a Hospital Management System, and identify practical methods or metrics for assessing them.

Conclusion

- McCall's Model categorizes software quality into Product Operation, Product Revision, and Product Transition, each with specific quality factors.
- The model includes 11 quality factors which cover both functional and non-functional aspects.
- McCall's Model provides a practical framework to assess and improve software quality in real-world systems.

Q & A

Thank You